



UNIVERSITAT POLITÈCNICA DE CATALUNYA

Escola Tècnica Superior d'Enginyeria de Telecomunicacions de Barcelona (ETSETB)

Practice 1

Grau en Enginyeria Física

Authors: Arnau Torret & Arnau Montull

Subject: SIMCOM

Date: October 4, 2026

1 Exercise 1

1.1 Question

Write a full program that adds one integer with one real. Be sure to use correct style (program name, no implicit typing, correct declarations). Add the user to enter the two numbers. Print the result in an external file.

1.2 Code

```
1 PROGRAM EX1
2   IMPLICIT NONE
3   INTEGER :: A
4   REAL    :: B, RESULT
5   INCLUDE './../chdir_to_code.inc'
6
7   PRINT *, 'Enter an integer:'
8   READ(*, *) A
9
10  PRINT *, 'Enter a real number:'
11  READ(*, *) B
12
13  RESULT = REAL(A) + B
14
15  PRINT '(A, F10.2)', 'The sum is: ', RESULT
16
17  OPEN(UNIT=10, FILE='ex1_results.txt', STATUS='REPLACE', ACTION='WRITE')
18  WRITE(10, '(A, F10.2)') 'The sum is: ', RESULT
19  CLOSE(10)
20
21 END PROGRAM EX1
```

1.3 Results

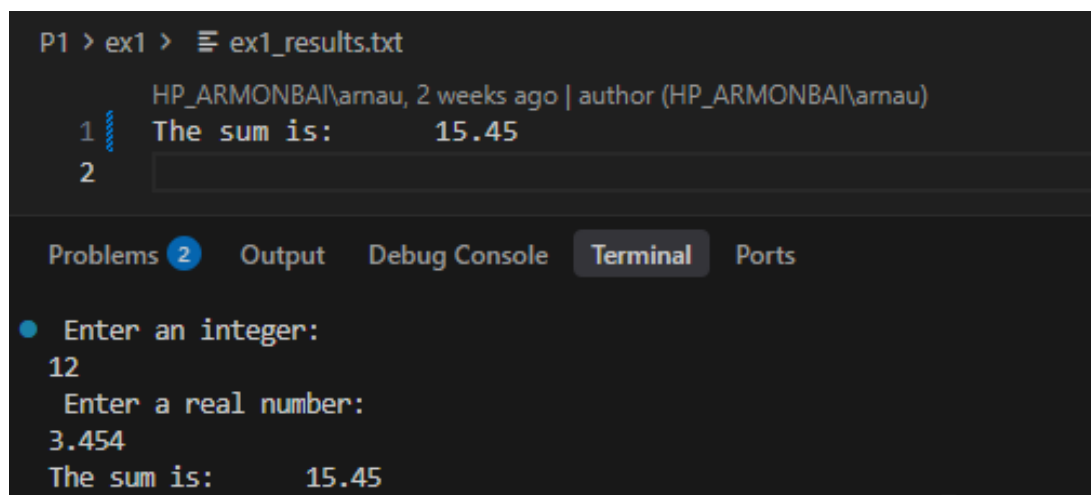


Figure 1: Screenshot of exercise 1

1.4 Discussion & Notes

As can be observed in the image, the program firstly ask the user for 1 integer and after the user insert it, the program asks for 1 real number. Finally, the program prints the result of the sum of the two numbers in an external file called `ex1_results.txt`.

2 Exercise 2

2.1 Question

Use the Leibniz formula to compute π :

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} \quad (1)$$

Compute until your result agrees to at least five decimal digits within the correct value (3.1415926535897932384626433...). This series converges very slowly so this can take a lot of iterations... Display a graph showing the evolution with k , i.e. $\pi(k)$ as a function of k .

2.2 Code

```
1 PROGRAM EX2
2   IMPLICIT NONE
3   DOUBLE PRECISION, PARAMETER :: pi = 3.14159265358979323846D0
4   DOUBLE PRECISION :: pi_aprox = 0.0D0, error = 1.0D0
5   INTEGER :: K = 0, sign_K = 1
6   INCLUDE '../chdir_to_code.inc'
7   OPEN(UNIT=10, FILE='ex2_results.dat', STATUS='REPLACE', ACTION='WRITE')
8
9   DO WHILE (ABS(error) > 1.0D-6)
10      pi_aprox = pi_aprox + sign_K * 4.0D0 / (2.0D0 * DBLE(K) + 1.0D0)
11      error = pi_aprox - pi
12
13      K = K + 1
14      WRITE(10, '(I8, 1X, ES16.8)') K, pi_aprox
15      sign_K = -sign_K
16   END DO
17
18   CLOSE(10)
19
20   PRINT *, 'The approximate value of pi for K =', K, 'is: ', pi_aprox
21 END PROGRAM EX2
```

2.3 Results

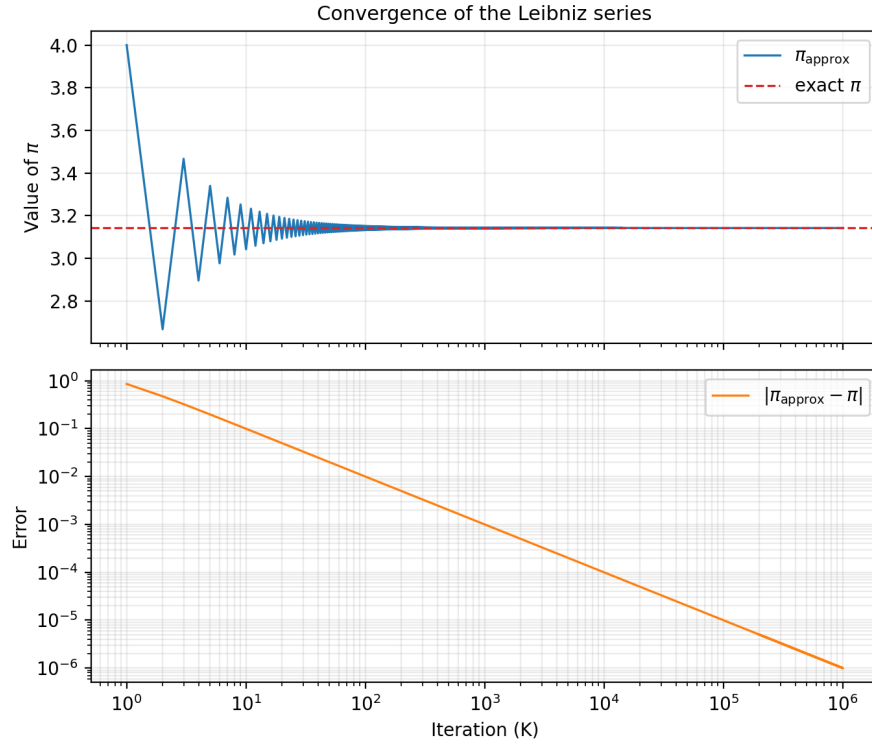


Figure 2: Output of exercise 2

2.4 Discussion & Notes

As we use the alternating Leibniz series to approximate π until the first five decimal places converge, we require an error tolerance one order of magnitude smaller ($|\varepsilon| < 10^{-6}$) to guarantee this accuracy. Alternatively, we could have truncated both the reference value and our approximation to five decimal digits and checked for zero difference. However, because the partial sums oscillate around the true value, we could have satisfied that condition prematurely simply by chance. For this reason, to ensure consistency and robustness in our results, we enforced an error threshold strictly below 10^{-6} .

Therefore, to obtain π with the desired accuracy, we need to compute the series until $K = 10^6$.

3 Exercise 3

3.1 Question

Use subroutines and/or functions to write a code to compute the Fourier integral of a given time-dependent correlation function such as $C(t) = \langle A(0) \cdot A(t) \rangle$. For instance, in molecular water simulations, spectral lineshapes $I(\omega)$ are usually found from:

$$I(\omega) \sim \frac{2Nq^2}{\pi\omega^2} S(\omega) \equiv \frac{2Nq^2}{\pi\omega^2} \int_0^\infty dt \langle \vec{v}_{H_i}(t) \cdot \vec{v}_{H_i}(0) \rangle \cos(\omega t) \quad (2)$$

Use the same idea to obtain $S(\omega)$ for $C(t)$, given that it is read from a known file, say file named `corr.dat` (corresponds to an oxygen velocity autocorrelation), with the usual two-column format $(t, C(t))$. You will find the file at the ATENEA database. Note that the time step used in `corr.dat` is of 0.0005 ps.

Find:

$$S(\omega) = \int_0^\infty dt C(t) \cdot \cos(\omega t) \quad (3)$$

3.2 Code

```
1 PROGRAM EX3
2   IMPLICIT NONE
3   INTEGER, PARAMETER :: npts = 2001
4   INTEGER, PARAMETER :: wmax = 500
5   REAL, DIMENSION(npts) :: C, t
6   REAL, DIMENSION(0:wmax) :: result
7   INTEGER :: i, w
8   INCLUDE '../chdir_to_code.inc'
9
10  OPEN(UNIT=10, FILE='ex3_data.dat', STATUS='OLD', ACTION='READ')
11  DO i = 1, npts
12    READ(10, *) t(i), C(i)
13  END DO
14  CLOSE(10)
15
16  CALL simpson(C, t, result)
17
18  OPEN(UNIT=20, FILE='ex3_results.dat', STATUS='REPLACE', ACTION='WRITE')
19  DO w = 0, wmax
20    WRITE(20, '(I8, 1X, ES16.8)') w, result(w)
21  END DO
22  CLOSE(20)
23
24  PRINT *, 'Results written to ex3_results.dat'
25  PRINT *, 'F(w=0) = ', result(0)
26
27 CONTAINS
28
29 SUBROUTINE simpson(C, t, result)
30   IMPLICIT NONE
31   REAL, DIMENSION(:), INTENT(IN) :: C, t
32   REAL, DIMENSION(0:), INTENT(OUT) :: result
33   INTEGER :: i, w, n
34   REAL :: h, s
35   REAL, ALLOCATABLE :: f(:)
36
37   n = SIZE(t)
38   h = t(2) - t(1)
39   ALLOCATE(f(n))
40
```

```

41      DO w = 0, 500
42          DO i = 1, n
43              f(i) = C(i) * COS(REAL(w) * t(i))
44          END DO
45
46          ! Composite Simpson (even number of intervals, step 2)
47          s = 0.0
48          DO i = 2, n - 1, 2
49              s = s + h / 3.0 * (f(i-1) + 4.0 * f(i) + f(i+1))
50          END DO
51          result(w) = s
52      END DO
53
54      DEALLOCATE(f)
55  END SUBROUTINE simpson
56
57  END PROGRAM EX3

```

Listing 1: Fortran implementation of composite Simpson's quadrature for the Fourier cosine transform.

3.3 Results

The input signal $C(t)$ and the resulting Fourier cosine transform spectrum $S(\omega)$ are illustrated in Figure 3.

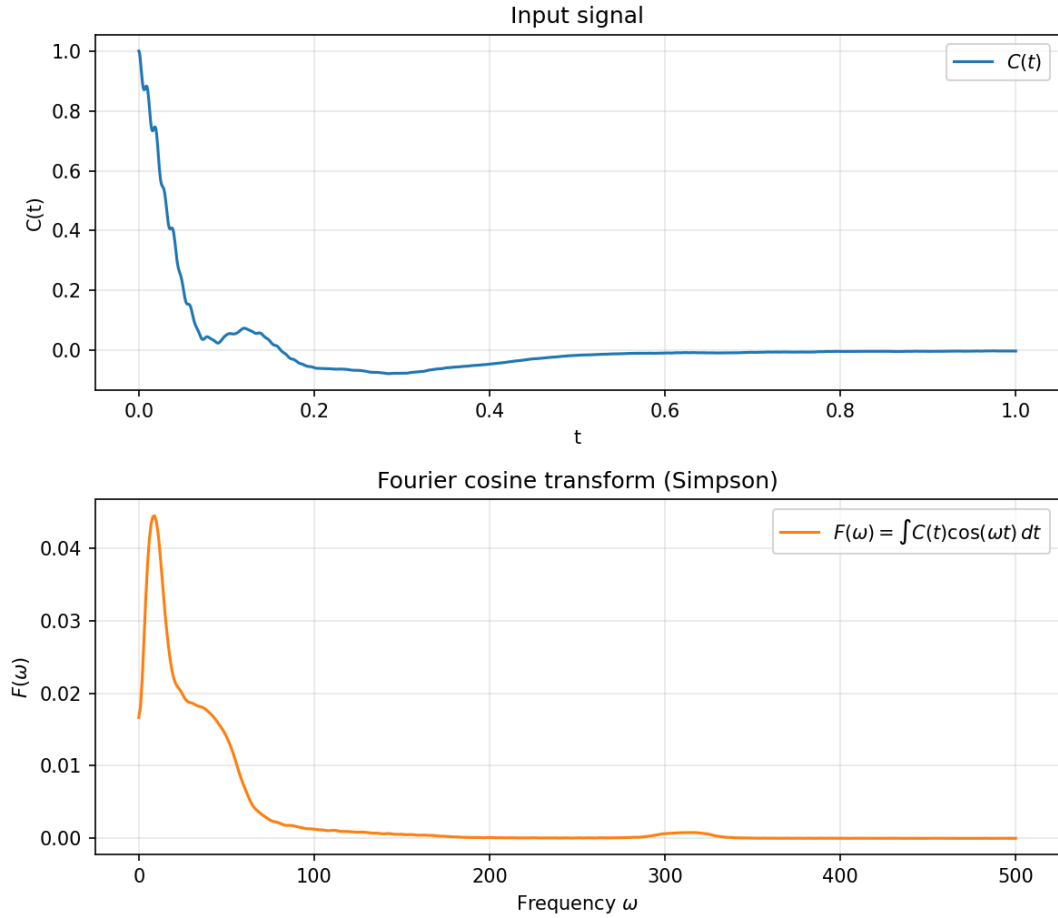


Figure 3: Top: Input time-domain signal $C(t)$ over $t \in [0, 1]$ ps. Bottom: Fourier cosine transform $S(\omega) = \int C(t) \cos(\omega t) dt$ computed via composite Simpson's rule for $\omega \in [0, 500]$ ps⁻¹.

3.4 Discussion & Notes

- The numerical integration was carried out using the composite Simpson's 1/3 rule. The dataset contains $N = 2001$ points with a uniform time step of $\Delta t = 0.0005$ ps, which yields an even number of intervals ($N - 1 = 2000$), strictly satisfying the condition required for the standard Simpson's rule.
- We implemented the integration routine into a dedicated Fortran subroutine, `simpson(C, t, result)`. For each frequency value in the range $\omega \in [0, 500]$ ps⁻¹, the integrand $f(t) = C(t) \cos(\omega t)$ is evaluated and integrated over the full time domain.
- The computed spectrum $S(\omega)$ matches the reference curve from the presentation: a dominant low-frequency peak located around $\omega \approx 10$ ps⁻¹, followed by a gradual decay and a smaller secondary local maximum near $\omega \approx 315$ ps⁻¹.

4 Exercise 4

4.1 Question

Compute integrals with the MC method, using random sampling.

Calculation of a surface: Idea: Use of the mean value theorem for integrals. The value of the integral is:

$$\int_a^b dx f(x) = (b-a)\langle f \rangle, \quad \text{where} \quad \langle f \rangle_N = \frac{1}{N} \sum_{i=1}^N f(x_i) \quad (4)$$

Evaluate the area enclosed between the curves $y = x^{-1}$ and $y = -x^{-1}$ within the range $x \in [-1, 1]$ and $y \in [-11, 11]$, represented in the figure. Note that the exact value is $A_{\text{exact}} = 13.5916$.

Procedure:

- A. Generate N points (x_i, y_i) at random. x_i belongs to a uniform distribution in the range $[-1, 1]$ and y_i belongs to a uniform distribution in the range $[-11, 11]$. (*Hint: You will need a good random number generator! Try `RANDOM_SEED`, works in most of Fortran versions*).
- B. Calculate the number of points N_{hit} inside the region of interest and estimate the area as follows:

$$A(N) = 2 \times 22 \times \frac{N_{\text{hit}}}{N} \quad (5)$$

- C. Run a number of different MC series (for instance, 3) with $N = \{10^2 - 10^7\}$. For each N , calculate the mean value of the area:

$$\langle A(N) \rangle = \frac{1}{M} \sum_{i=1}^M A_i(N) \quad (6)$$

and the standard deviation σ (variance defined as σ^2):

$$\langle \sigma(N) \rangle = \sqrt{\langle (A(N) - A_{\text{exact}})^2 \rangle} \quad (7)$$

- D. Finally plot $A(N)$ and $\sigma(N)$.

4.2 Code

```
1 PROGRAM EX4
2   IMPLICIT NONE
3   INTEGER, PARAMETER :: nmax = 10**7
4   INTEGER, PARAMETER :: nruns = 3
5   REAL, DIMENSION(nmax) :: x, y
6   REAL :: A, A_average, A_exact, A_deviation, A_deviation_average
7   INTEGER :: N, J, i
8   INCLUDE '../chdir_to_code.inc'
9
10  A_exact = 4.0 * (1.0 + LOG(11.0))
11
12  OPEN(UNIT=10, FILE='ex4_results.dat', STATUS='REPLACE', ACTION='WRITE')
13  DO i = 2, 7, 1
14    N = 10**i
15    WRITE(*, *) 'N = ', N
16    A_average = 0.0
17    A_deviation = 0.0
18    DO J = 1, nruns
```

```

19      CALL RANDOM_NUMBER(x(1:N))
20      CALL RANDOM_NUMBER(y(1:N))
21      x(1:N) = x(1:N) * 2.0 - 1.0
22      y(1:N) = y(1:N) * 22.0 - 11.0
23      CALL calculate_area(x, y, N, A)
24      A_average = A_average + A
25      A_deviation = A_deviation + (A - A_exact)**2
26  END DO
27  A_average = A_average / REAL(nruns)
28  A_deviation_average = SQRT(A_deviation / REAL(nruns))
29  WRITE(10, '(I8, 1X, ES16.8, 1X, ES16.8)') N, A_average, A_deviation_average
30  END DO
31  CLOSE(10)
32
33  PRINT *, 'Results written to ex4_results.dat'
34  PRINT *, 'Exact area = ', A_exact
35
36  CONTAINS
37
38  SUBROUTINE calculate_area(x, y, N, A)
39      IMPLICIT NONE
40      INTEGER, INTENT(IN) :: N
41      REAL, DIMENSION(:), INTENT(IN) :: x, y
42      REAL, INTENT(OUT) :: A
43      INTEGER :: k, Nh
44
45      Nh = 0
46      DO k = 1, N
47          IF (x(k) /= 0.0) THEN
48              IF (ABS(y(k)) <= ABS(1.0 / x(k))) THEN
49                  Nh = Nh + 1
50              END IF
51          END IF
52      END DO
53      A = 44.0 * REAL(Nh) / REAL(N)
54  END SUBROUTINE calculate_area
55
56  END PROGRAM EX4

```

4.3 Results

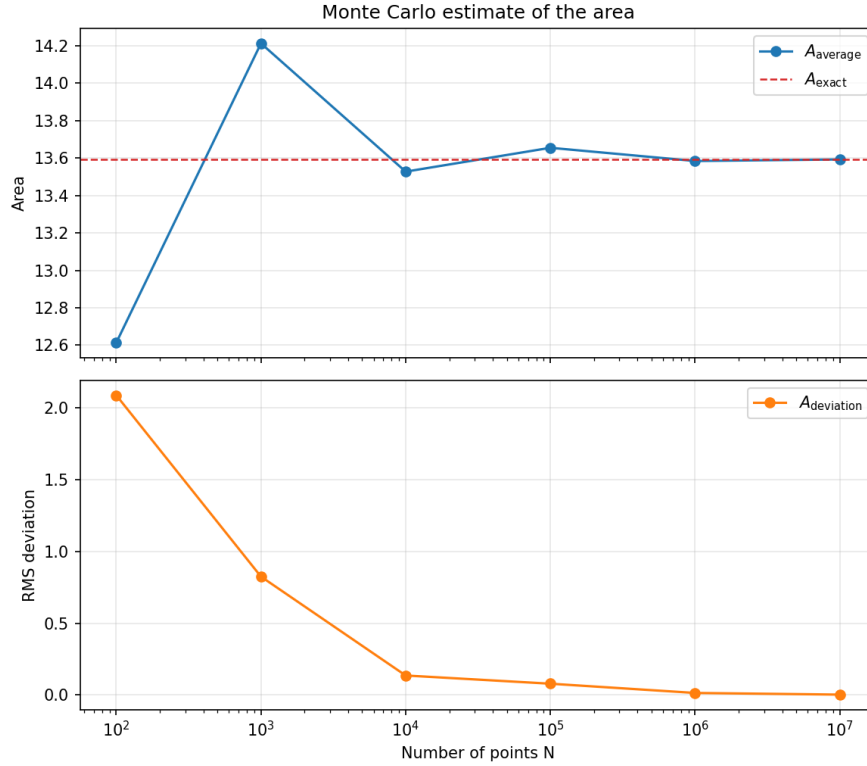


Figure 4: Output of exercise 4

4.4 Discussion & Notes

- **Simulation setup and method:** We tested sample sizes from $N = 100$ up to 10^7 , running each test 3 times. In each run, we picked N random points (x, y) inside a rectangle of width 2 and height 22 (total area = 44). A point counts as a “hit” if it lands under the curve ($|y| \leq |1/x|$). We then estimated the area using:

$$\text{Area} = 44 \times \frac{\text{Hits}}{N}$$

Finally, we averaged the 3 runs and calculated how far the result was from the true value, $A_{\text{exact}} \approx 13.5916$.

- **Code details:** To avoid dividing by zero when calculating $1/x$, the code checks `IF (x /= 0.0)` before testing a point; skipping this single line does not change the calculated area.
- **Sweet spot:** As N increases, the estimated area converges toward the analytical value (13.5916), with the error decreasing as $1/\sqrt{N}$, as expected for Monte Carlo integration:
 - At $N = 10^2$, the relative error is substantial ($\approx 15\%$).
 - By $N = 10^4$, the error drops to approximately 1%.
 - At $N = 10^6$, the error reaches $\approx 0.1\%$, providing high precision within seconds.
 - At $N = 10^7$, the estimate (13.592) matches the true value to three decimal places ($< 0.02\%$ error).

Since no target accuracy was explicitly specified, $N = 10^6$ serves as the optimal trade-off: it achieves sub-percent accuracy ($\sim 0.1\%$) with minimal runtime, whereas scaling to $N = 10^7$ increases computation time by a factor of 10 for only a modest threefold gain in precision.

5 Exercise 5

5.1 Question

Consider an harmonic oscillator of mass $m = 200$ g and force constant $k = 2$ N/m. Integrate numerically the equation of motion and compare the results of positions and velocities with the exact analytical solution. Compute also the kinetic, potential and total energies.

Initial position and velocity: $x_0 = 5$ cm, $v_0 = 0$.

Integration algorithms and timesteps:

- Euler: $\Delta t = 0.001$ s, $\Delta t = 0.01$ s, $\Delta t = 0.02$ s
- Euler predictor-corrector: $\Delta t = 0.02$ s
- Verlet: $\Delta t = 0.02$ s

5.2 Code

```
1 PROGRAM EX5
2   IMPLICIT NONE
3   REAL :: m = 0.200, k = 2
4   REAL :: t_max = 10.0, x_0 = 0.05, v_0 = 0.0, t_0 = 0.0
5
6   CALL EULER(m, k, t_max, 0.001, x_0, v_0, t_0, "euler_001.dat")
7   CALL EULER(m, k, t_max, 0.010, x_0, v_0, t_0, "euler_01.dat")
8   CALL EULER(m, k, t_max, 0.020, x_0, v_0, t_0, "euler_02.dat")
9   CALL EULER_PREDICTOR(m, k, t_max, 0.020, x_0, v_0, t_0, "euler_predictor.dat")
10  CALL VERLET(m, k, t_max, 0.020, x_0, v_0, t_0, "verlet.dat")
11
12  PRINT *, "Wrote euler_*.dat, euler_predictor.dat, verlet.dat"
13
14  CONTAINS
15
16  SUBROUTINE EULER(m, k, t_max, dt, x_0, v_0, t_0, fname)
17    IMPLICIT NONE
18    REAL, INTENT(IN) :: m, k, t_max, dt, x_0, v_0, t_0
19    CHARACTER(LEN=*) , INTENT(IN) :: fname
20    REAL :: x, v, t, a, ek, ep, et
21    INTEGER :: i, n, u
22
23    n = INT((t_max - t_0) / dt)
24    x = x_0
25    v = v_0
26    t = t_0
27    u = 20
28
29    OPEN(UNIT=u, FILE=fname, STATUS="REPLACE", ACTION="WRITE")
30    WRITE(u, '(A)') "# t x v Ek Ep ET"
31    WRITE(u, '(6ES16.8)') t, x, v, 0.5*m*v**2, 0.5*k*x**2, 0.5*(m*v**2 + k*x**2)
32
33    DO i = 1, n
34      a = -(k/m) * x
35      x = x + v*dt + 0.5*a*dt**2
36      v = v + a*dt
37      t = t + dt
38      ek = 0.5*m*v**2
39      ep = 0.5*k*x**2
40      et = ek + ep
41      WRITE(u, '(6ES16.8)') t, x, v, ek, ep, et
```

```

42     END DO
43     CLOSE(u)
44 END SUBROUTINE EULER
45
46 SUBROUTINE VERLET(m, k, t_max, dt, x_0, v_0, t_0, fname)
47     IMPLICIT NONE
48     REAL, INTENT(IN) :: m, k, t_max, dt, x_0, v_0, t_0
49     CHARACTER(LEN=*), INTENT(IN) :: fname
50     REAL :: x, v, t, a, a_new, ek, ep, et
51     INTEGER :: i, n, u
52
53     n = INT((t_max - t_0) / dt)
54     x = x_0
55     v = v_0
56     t = t_0
57     a = -(k/m) * x
58     u = 20
59
60     OPEN(UNIT=u, FILE=fname, STATUS="REPLACE", ACTION="WRITE")
61     WRITE(u, '(A)') "# t x v Ek Ep ET"
62     WRITE(u, '(6ES16.8)') t, x, v, 0.5*m*v**2, 0.5*k*x**2, 0.5*(m*v**2 + k*x**2)
63
64     DO i = 1, n
65         x = x + v*dt + 0.5*a*dt**2
66         a_new = -(k/m) * x
67         v = v + 0.5*(a + a_new)*dt
68         a = a_new
69         t = t + dt
70         ek = 0.5*m*v**2
71         ep = 0.5*k*x**2
72         et = ek + ep
73         WRITE(u, '(6ES16.8)') t, x, v, ek, ep, et
74     END DO
75     CLOSE(u)
76 END SUBROUTINE VERLET
77
78 SUBROUTINE EULER_PREDICTOR(m, k, t_max, dt, x_0, v_0, t_0, fname)
79     IMPLICIT NONE
80     REAL, INTENT(IN) :: m, k, t_max, dt, x_0, v_0, t_0
81     CHARACTER(LEN=*), INTENT(IN) :: fname
82     REAL :: x, v, t, a, x_p, a_p, ek, ep, et
83     INTEGER :: i, n, u
84
85     n = INT((t_max - t_0) / dt)
86     x = x_0
87     v = v_0
88     t = t_0
89     u = 20
90
91     OPEN(UNIT=u, FILE=fname, STATUS="REPLACE", ACTION="WRITE")
92     WRITE(u, '(A)') "# t x v Ek Ep ET"
93     WRITE(u, '(6ES16.8)') t, x, v, 0.5*m*v**2, 0.5*k*x**2, 0.5*(m*v**2 + k*x**2)
94
95     DO i = 1, n
96         a = -(k/m) * x
97         x_p = x + v*dt + 0.5*a*dt**2
98         a_p = -(k/m) * x_p
99         a = 0.5*(a_p + a)
100        x = x + v*dt + 0.5*a*dt**2
101        v = v + a*dt
102        t = t + dt
103        ek = 0.5*m*v**2
104        ep = 0.5*k*x**2

```

```

105      et = ek + ep
106      WRITE(u, '(6ES16.8)') t, x, v, ek, ep, et
107  END DO
108  CLOSE(u)
109  END SUBROUTINE EULER_PREDICTOR
110
111  END PROGRAM EX5

```

Listing 2: Fortran implementation of explicit Euler, Euler Predictor-Corrector, and Velocity-Verlet algorithms.

5.3 Results

The equations of motion for the simple harmonic oscillator ($m = 0.200$ kg, $k = 2$ N/m, $x_0 = 0.05$ m, $v_0 = 0$ m/s) were integrated up to $t_{\max} = 10$ s using the explicit Euler scheme ($\Delta t \in \{0.001, 0.01, 0.02\}$ s), the Euler Predictor-Corrector scheme ($\Delta t = 0.02$ s), and the Velocity-Verlet algorithm ($\Delta t = 0.02$ s).

Figure 5 displays the simulated trajectories $x(t)$ compared against the analytical solution $x(t) = x_0 \cos(\omega_0 t)$, highlighting the effect of the step size Δt on Euler's method as well as the comparison across all three integrators at $\Delta t = 0.02$ s.

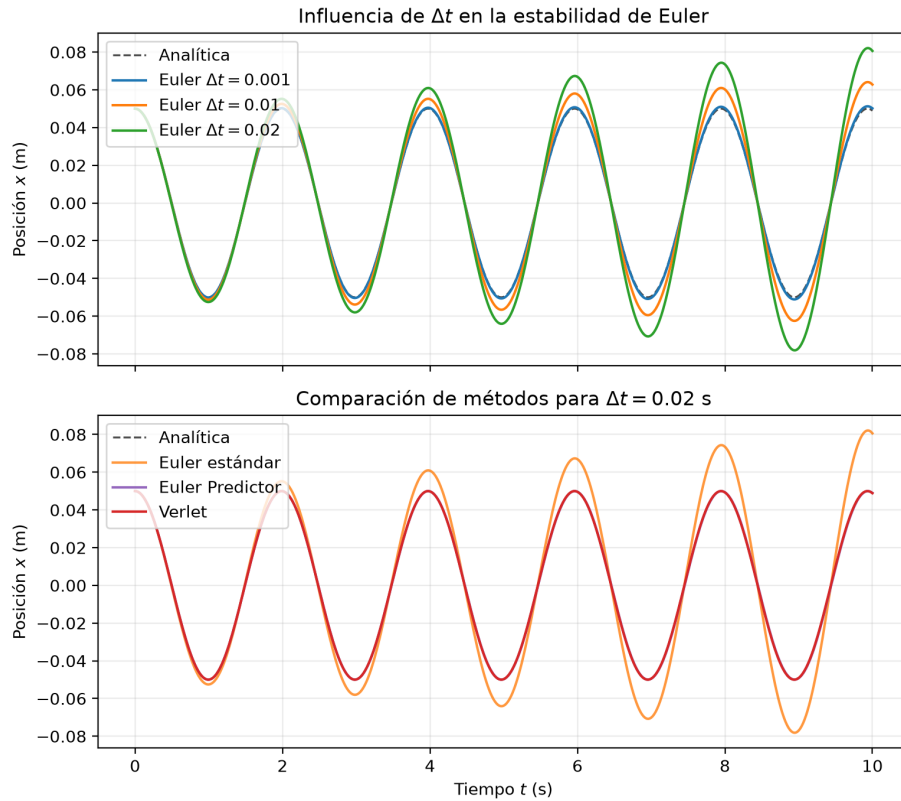


Figure 5: Oscillator position $x(t)$ over time. Top: Influence of $\Delta t \in \{0.001, 0.01, 0.02\}$ s on the standard Euler method. Bottom: Comparison between standard Euler, Euler Predictor, and Velocity-Verlet for $\Delta t = 0.02$ s against the analytical solution.

The corresponding position errors $|x_{\text{num}}(t) - x_{\text{exact}}(t)|$ are shown on a logarithmic scale in Figure 6.

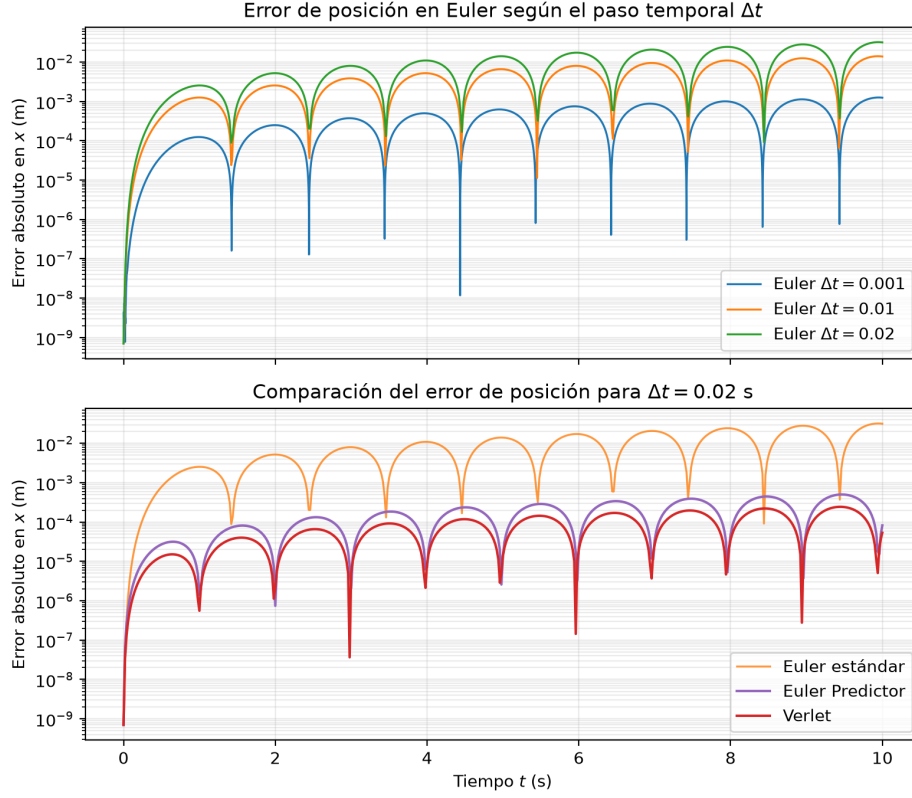


Figure 6: Absolute error in position $|x_{\text{num}} - x_{\text{exact}}|$ (logarithmic scale). Top: Error in Euler's method as a function of Δt . Bottom: Numerical error comparison among the three algorithms for $\Delta t = 0.02$ s.

Figure 7 presents the individual energy balances (kinetic E_k , potential E_p , and total mechanical energy E_{tot}) for each algorithm at $\Delta t = 0.02$ s, comparing them with the exact reference $E_0 = 2.5 \times 10^{-3}$ J.

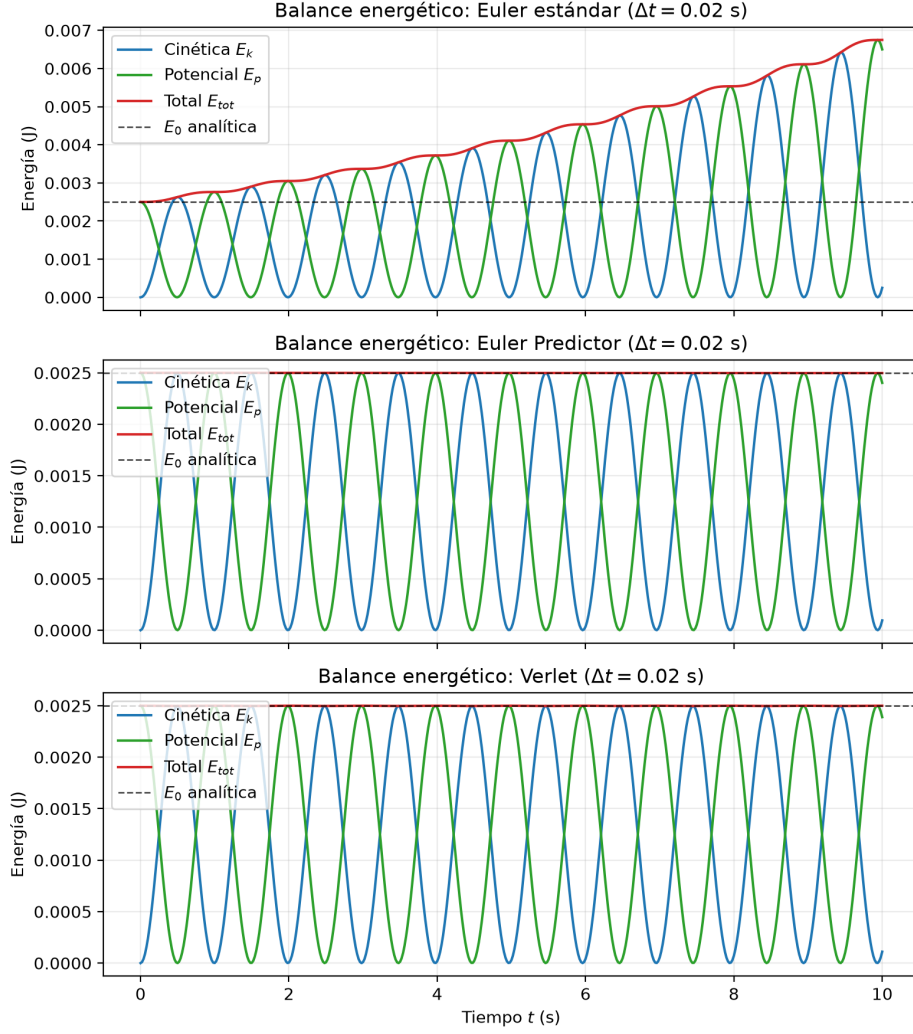


Figure 7: Kinetic, potential, and total energy evolution at $\Delta t = 0.02$ s. Top: Standard Euler. Middle: Euler Predictor. Bottom: Velocity-Verlet.

A direct comparison of the total energy conservation $E_{tot}(t)$ among all three algorithms is displayed in Figure 8. The absolute energy errors $|E_{tot}(t) - E_0|$ across step sizes and integration schemes are presented in Figure 9.

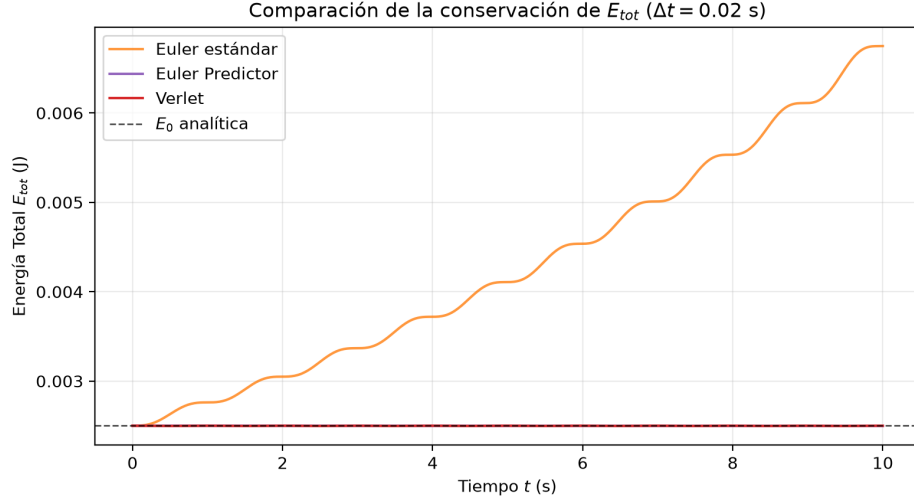


Figure 8: Comparison of total mechanical energy $E_{tot}(t)$ conservation between standard Euler, Euler Predictor, and Velocity-Verlet at $\Delta t = 0.02$ s.

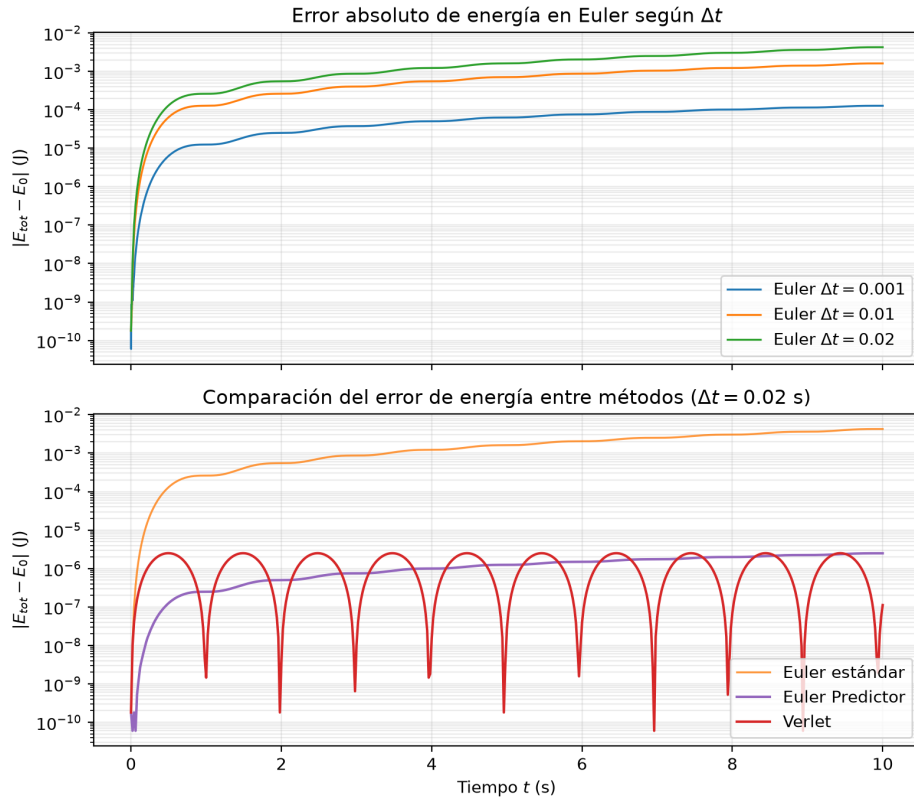


Figure 9: Absolute error in total energy $|E_{tot} - E_0|$ (logarithmic scale). Top: Effect of time step Δt in Euler's method. Bottom: Comparison across the three methods for $\Delta t = 0.02$ s.

5.4 Discussion & Notes

- **Influence of Time Step (Δt) on Euler's Method:** As observed in both the trajectory and error curves, the explicit Euler method is highly sensitive to the choice of integration step size. For $\Delta t = 0.02$ s and $\Delta t = 0.01$ s, the algorithm suffers from an intrinsic numerical instability that artificially amplifies the oscillation amplitude over time. Decreasing the step size to $\Delta t = 0.001$ s significantly mitigates this growth and reduces the position error by over an order of magnitude ($\sim 10^{-3}$ m compared to $\sim 3 \times 10^{-2}$ m for $\Delta t = 0.02$ s), though a gradual cumulative phase lag remains.
- **Performance Comparison Across Algorithms ($\Delta t = 0.02$ s):** Setting the step size to $\Delta t = 0.02$ s clearly highlights the advantages of higher-order schemes over standard Euler:
 - *Standard Euler:* Demonstrates unbounded error growth reaching nearly 60% of the initial amplitude by $t = 10$ s, making it unsuitable for long-term oscillatory dynamics.
 - *Euler Predictor-Corrector:* By averaging the predicted and current accelerations $\frac{1}{2}(a_n + a_{n+1}^{\text{pred}})$, the leading first-order truncation error terms cancel out, lowering the trajectory error down to $\sim 10^{-4}$ m.
 - *Velocity-Verlet:* Yields the closest agreement with the analytical trajectory $x(t) = x_0 \cos(\omega_0 t)$, keeping the numerical error bounded within 10^{-5} m to 10^{-4} m throughout the entire simulation interval.
- **Mechanical Energy Conservation and Symplecticity:** The exact mechanical energy of the harmonic oscillator is constant:

$$E_0 = \frac{1}{2} k x_0^2 = \frac{1}{2} (2 \text{ N/m}) (0.05 \text{ m})^2 = 2.5 \times 10^{-3} \text{ J} \quad (8)$$

Examining the energy balances illustrates the qualitative differences between the methods:

- In standard Euler, the phase shift between position and velocity causes a continuous, unphysical energy injection into the system. The total energy E_{tot} undergoes an almost linear secular drift, nearly tripling to $E_{\text{tot}} \approx 6.7 \times 10^{-3}$ J by $t = 10$ s.
- The Euler Predictor-Corrector scheme successfully bounds the total energy close to the nominal value, though without formal symplecticity it may exhibit slow drift over much longer integration times.
- Velocity-Verlet confirms its nature as a time-reversible, symplectic integrator. The kinetic and potential energies alternate symmetrically each quarter period, and total energy oscillates within tight bounds around E_0 without secular drift, making it the algorithm of choice for physical simulations and molecular dynamics.